BULUT MİMARİSİ VE ALGORİTMİK SİSTEM MÜHENDİSLİĞİ ARAŞTIRMA RAPORU

Tek Kişilik Algoritmik Trading Panosu İçin 2026 İtibarıyla En Uygun Altyapı Mimarisi

GitHub Actions JSON Çıktısından Canlı İzleme ve İcra Karar Panosuna Geçiş: Statik Site (React/Svelte), Streamlit/Dash/Panel, FastAPI + SQLite/Postgres, Mobil PWA ve Telegram Bot Arayüzlerinin Maliyet, Güvenlik ve Bakım Yükü Analizi

Bireysel bir algoritmik yatırımcının (solo quant developer) en değerli kaynağı zamanıdır. Strateji araştırması ve risk yönetimine ayrılması gereken zamanın sunucu çökmeleri, veritabanı yedeklemeleri, SSL sertifikası yenilemeleri veya konteyner soğuk başlangıç (cold-start) sorunlarıyla harcanması, sistemin sürdürülebilirliğini doğrudan tehdit eder. Bu rapor, halihazırda **GitHub Actions üzerinde birkaç saatte bir çalışan ve portföy/sinyal durumunu JSON dosyası olarak üreten** bir mimariden yola çıkarak, gerçek hesap verilerini güvenle gösterecek, canlı fiyatları takip edecek ve mobilde kesintisiz çalışacak 5 aday altyapıyı 2026 yılı bulut sağlayıcı kısıtları çerçevesinde karşılaştırmaktadır.

Mimari Seçenek	2026 Aylık Maliyet	Temel Avantaj (Güçlü Yön)	Temel Kısıt / Darboğaz
(a) Statik Site (React/Vite veya Svelte + GH Pages / CF Pages)	**$0 / ay** (CF Pages + Access ile; GH Pro $4/ay)	Sıfır sunucu bakımı, CDN hızında anlık açılış (~0.3 sn), tarayıcıdan doğrudan borsa WebSocket bağlantısı.	GitHub Pages Free özel depolarda çalışmaz ve kimlik doğrulama sunmaz; Cloudflare Zero Trust şarttır.
(b) Python Veri Panosu (Streamlit / Dash / Panel)	**$0 / ay** (Uyku modlu) veya **$2–$7 / ay** (7/24)	Sadece Python ile hızlı geliştirme (ortalama 14 gün), Pandas/NumPy koduyla doğrudan entegrasyon.	Ücretsiz katmanlarda (Streamlit, Render, HF Spaces) 15 dk–48 saat hareketsizlikte uyku ve 25–60 sn uyanma süresi.
(c) Tam Yığın (FastAPI + SQLite/Postgres + Ön Yüz)	**$1.94 – $7.00 / ay** (Fly.io / Render + Neon)	Gerçek zamanlı özel bakiye (Private WS), ilişkisel işlem geçmişi, kurumsal JWT/OAuth2 güvenliği.	En yüksek geliştirme ve bakım yükü (~3.8–4.5 saat/ay); iki ayrı katmanın (API + UI) ve şema göçlerinin yönetimi.
(d) Mobil PWA (Progressive Web App) Kurulumu	**$0 / ay** ((a) veya (c) üzerine ek katman)	App Store ücreti ($99/yıl) olmadan iOS/Android ana ekran ikonu, tam ekran görünüm ve Web Push bildirimi.	iOS Safari'de arka plan senkronizasyonu kısıtlıdır; Web Push yalnızca ana ekrana eklenince çalışır.
(e) Telegram Bot Arayüzü (+ Uyguladım / Atladım Butonu)	**$0 / ay** (Cloudflare Workers + D1 Free)	Sıfır ön yüz kodu, 0.2 sn anlık bildirim, `InlineKeyboardMarkup` ile tek tıkla karar kaydı (Webhook).	Çok boyutlu tablolar, korelasyon ısı haritaları ve interaktif mum grafikleri için sohbet ekranı yetersiz kalır.

Rapor Tarihi: 26 Ekim 2026 **Mevcut Veri Kaynağı:** GitHub Actions (Saatlik/Periyodik JSON Üretimi) **Hedef Profil:** Tek Kişilik Bakım (Solo Quant)

1. Mevcut Durum Analizi: GitHub Actions JSON İş Akışının Avantajları ve Yapısal Sınırları

Halihazırda çalışan mimaride **GitHub Actions**, belirli aralıklarla (örneğin birkaç saatte bir `schedule` cron tetikleyicisiyle veya manuel `workflow_dispatch` ile) Python strateji betiğini çalıştırmakta, borsalardan (Binance, BIST/VİOP veri sağlayıcısı veya Forex API) verileri çekip hesaplamaları yapmakta ve sonuçları bir `state.json` / `portfolio.json` dosyası olarak üretmektedir. Tek kişilik bir algoritmik yatırım operasyonu için bu başlangıç noktası **son derece doğru bir temeldir**; çünkü GitHub Actions genel (public) depolarda sınırsız, özel (private) depolarda ise ücretsiz katmanda (GitHub Free) **aylık 2,000 dakika işlem süresi ve 500 MB depolama alanı** sunarak hesaplama (compute) maliyetini sıfırlamaktadır (GitHub Product Plans, 2026).

Ancak 2026 yılı itibarıyla üretim ortamındaki uygulayıcı deneyimlerinin gösterdiği üzere, yalnızca GitHub Actions ve ham JSON çıktısına güvenmenin **üç kritik operasyonel darboğazı** bulunmaktadır:

1. **GitHub Actions Cron Gecikmesi (Schedule Drift):** GitHub'ın kendi `on: schedule` zamanlayıcısı yüksek yük altında garantili gerçek zamanlı çalışmaz; açık kaynak portföy izleme projelerinde yapılan ölçümlerde GitHub cron tetikleyicilerinin yoğun saatlerde **15 dakikadan birkaç saate kadar gecikebildiği veya tamamen atlanabildiği** belgelenmiştir (Richfolio GitHub Actions Benchmark). Bu nedenle zamanlamayı GitHub'ın iç cron'una bırakmak yerine, ücretsiz bir **Cloudflare Worker Cron Trigger** üzerinden GitHub REST API'sine `workflow_dispatch` isteği göndermek gecikmeyi saniyeler düzeyine indirir.
2. **Çift Yönlü Durum (State) Eksikliği:** GitHub Actions birkaç saatte bir JSON ürettiğinde veri akışı *tek yönlüdür* (GitHub → Kullanıcı). Yatırımcı gelen bir sinyale "Uyguladım (Executed)" veya "Atladım (Skipped)" yanıtı verdiğinde, bu kararın geri yazılıp bir sonraki GitHub Actions çalışmasında gerçek portföy durumuyla uzlaştırılması (reconciliation) için hafif bir geri yazma (write-back) katmanına ihtiyaç vardır.
3. **Saatlik JSON ile Saniyelik Piyasa Fiyatı Arasındaki Boşluk:** JSON dosyası 2–4 saatte bir güncellendiğinden, dosya içindeki `unrealized_pnl` veya `mark_price` değerleri birkaç dakika içinde bayatlar (stale). Panonun, temel pozisyon büyüklüklerini (`position_qty`, `entry_price`, `stop_price`) saatlik JSON'dan okurken, **anlık fiyatları canlı olarak güncellemesi** gerekir.

Aşağıdaki bölümlerde, bu üç darboğazı çözerken tek bir kişinin bakım yükünü (DevOps) minimumda tutan beş mimari seçenek tüm teknik detaylarıyla incelenmektedir.

---

2. Beş Aday Altyapı Mimarisinin Derinlemesine Teknik Analizi (a – e)

2.1. Seçenek (a): Statik Site Mimarisi (React/Vite veya Svelte + GitHub Pages vs. Cloudflare Pages)

Statik site mimarisinde ön yüz (React + Vite veya derleme boyutu çok daha küçük ve reaktif olan **Svelte / SvelteKit Static Adapter** ile yazılmış tek sayfa uygulaması — SPA), derleme (build) sonrasında saf HTML, CSS ve JavaScript dosyalarına dönüştürülür. Tarayıcıda açılan uygulama, GitHub Actions'ın ürettiği `portfolio.json` dosyasını `fetch()` ile çeker ve grafik kütüphaneleriyle (örneğin TradingView *Lightweight Charts* veya Apache ECharts) ekrana çizer.

- **Aylık Maliyet ve Ücretsiz Katman Sınırları:**
  - **GitHub Pages:** GitHub Pages Resmi Limitleri uyarınca site boyutu azami **1 GB**, aylık bant genişliği (soft limit) **100 GB** ve saatlik derleme sınırı **10 build/saat**'tir. Ancak en kritik kısıt şudur: **GitHub Free planında GitHub Pages yalnızca herkese açık (Public) depolarda ücretsizdir!** Kodunuzu ve JSON verinizi özel (Private) bir depoda tutarak GitHub Pages yayınlamak için **GitHub Pro ($4/ay)** gerekir. Daha da önemlisi, GitHub Pro alsanız bile *özel depodan yayınlanan bir GitHub Pages sitesinin URL'si internetteki herkese açıktır*; URL bazlı erişim kontrolü (Private Pages) yalnızca kurumsal **GitHub Enterprise Cloud ($21/kullanıcı/ay)** planında sunulmaktadır (GitHub Enterprise Docs).

    - **Cloudflare Pages (Önerilen Statik Alternatif):** Özel (Private) GitHub deponuza doğrudan bağlanır, ücretsiz katmanda **sınırsız bant genişliği**, aylık **500 derleme (build)** ve küresel uç (edge) CDN sunar; maliyeti **$0/ay**'dır (Cloudflare Free Tier Analizi, Haziran 2026; BOSC Pipeline).

- **Kimlik Doğrulama (Gerçek Hesap Verisinin Korunması):** Gerçek hesap bakiyesi, açık pozisyonlar ve likidasyon fiyatları asla herkese açık bir GitHub Pages URL'sinde düz metin olarak yayınlanmamalıdır. Statik sitede gerçek hesap verisini korumanın 2026 itibarıyla iki geçerli yöntemi vardır:
    1. **Cloudflare Zero Trust Access (Tamamen Ücretsiz — En İyi Uygulama):** Sitenizi Cloudflare Pages üzerinde barındırıp önüne tek tıkla **Cloudflare Access** koyduğunuzda, Cloudflare ücretsiz katmanda **50 kullanıcıya kadar ($0/ay)** kurumsal düzeyde Sıfır Güven (Zero Trust) kimlik doğrulaması sağlar (Display.dev, Ekim 2026; Cloudflare Static Auth). Siz kendi e-posta adresinizi (Google/GitHub OAuth veya tek kullanımlık e-posta PIN'i) beyaz listeye alırsınız; kimlik doğrulaması geçilmeden ne HTML sayfası ne de `portfolio.json` dosyası Cloudflare CDN uçlarından dışarı sızar.
    2. **İstemci Taraflı AES-256-GCM Şifreleme (Sıfır Sunucu Alternatifi):** Eğer mutlaka ücretsiz GitHub Pages kullanılacaksa, GitHub Actions JSON dosyasını yayınlamadan önce bir parola (GitHub Secret) ve PBKDF2 kullanarak **AES-256-GCM ile şifreler** (`encrypted_state.bin`). Statik React/Svelte sayfası açıldığında sizden parolanızı ister ve tarayıcının yerleşik **Web Crypto API**'si (`window.crypto.subtle.decrypt`) ile veriyi yalnızca RAM üzerinde çözer.

- **Canlı Fiyat Güncellemesi:** Statik sitenin en büyük sürpriz gücü budur. Pano statik olsa bile, kullanıcının tarayıcısında çalışan JavaScript kodu doğrudan **Binance genel (public) WebSocket akışına** (`wss://fstream.binance.com/ws/!markPrice@arr@1s`) bağlanabilir! Böylece GitHub Actions'ın 3 saat önce ürettiği JSON'dan `BTCUSDT: +0.5 BTC, Giriş: $88,400` verisi okunurken, saniyelik fiyat tarayıcı WebSocket'inden alınır ve **anlık PnL kuruşu kuruşuna canlı hesaplanır**. BIST/VİOP ve Forex için ise tarayıcıdan 15–30 saniyede bir hafif REST/JSON uç noktası yoklanır.
- **API Anahtarlarının Güvenliği:** Borsa özel API anahtarları (Binance API Key/Secret, aracı kurum şifreleri) yalnızca **GitHub Actions Secrets** kasasında durur; statik siteye yalnızca üretilmiş sonuç JSON'u gittiği için ön yüzde anahtar sızıntısı riski **sıfırdır**.
- **Bakım Yükü ve Mobil Kullanım:** Sunucu, işletim sistemi veya veritabanı olmadığı için bakım yükü **~0.5–0.8 saat/ay** ile neredeyse sıfırdır. Tailwind CSS ile yazılmış bir Svelte/React arayüzü mobilde kusursuz ve anlık (~0.3 sn) açılır.

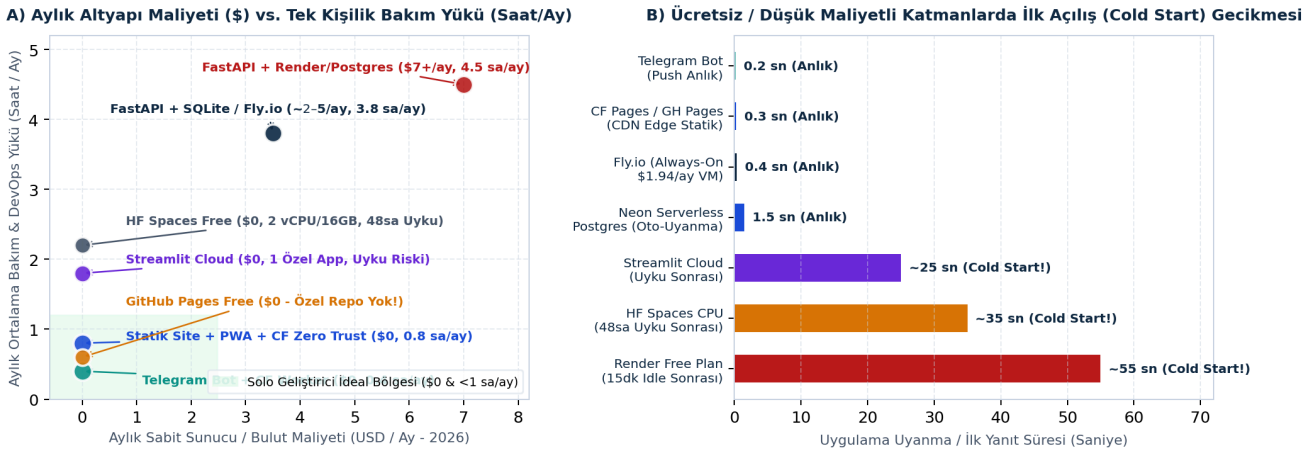
2.2. Seçenek (b): Python Veri Uygulamaları — Streamlit / Plotly Dash / HoloViz Panel (Streamlit Cloud, HF Spaces, Fly.io, Render)

Algoritmik strateji kodu zaten Python ile yazıldığı için, ön yüzde JavaScript (React/Svelte) öğrenmek yerine doğrudan Python kütüphaneleriyle (**Streamlit**, **Plotly Dash** veya **HoloViz Panel**) pano oluşturmak solo geliştiriciler arasında çok popülerdir. 2026 yılında yapılan 120 mühendislik ekibi araştırması (Johal.in Pandas Dashboard Benchmark, Mayıs 2026), ön yüz deneyimi olmayan geliştiricilerin Streamlit ile ortalama **14 günde** üretim panosu çıkarabildiğini (Dash için 42 gün), ancak **p99 gecikmesinde Dash 2.17+'nın React tabanlı mimarisiyle Streamlit'ten 210 ms daha hızlı olduğunu** ve **HoloViz Panel 1.4+'ün bellek (RAM) tüketimini %38 azalttığını** göstermektedir.

- **Bulut Sağlayıcıların 2026 Maliyet ve Ücretsiz Katman Sınırları:**
    - **Streamlit Community Cloud ($0/ay):** Herkese açık (public) uygulamalar sınırsızdır ancak **hesap başına yalnızca 1 adet Özel (Private) uygulamaya** ücretsiz izin verir (Streamlit Docs). Kaynak sınırı uygulama başına **~1 GB RAM (2.7 GB hard limit)** düzeyindedir. En büyük operasyonel sorun **Uyku Modu (App Hibernation)** kuralıdır: Uygulama belirli bir süre ziyaret edilmediğinde uykuya dalar ve sabah telefondan açtığınızda **"Zzz... Your app is in the oven"** ekranıyla **20 ila 45 saniye** uyanmasını beklersiniz (Streamlit Community Forum, 2025; DanubeLabs, Ekim 2026).

- **Hugging Face Spaces ($0/ay):** Ücretsiz "CPU Basic" katmanı **2 vCPU ve 16 GB RAM** gibi devasa bir donanım sunar ve özel (Private) Space oluşturmayı destekler (HF Spaces Overview; Codex, Mayıs 2026). Ancak ücretsiz katmanda **48 saat hareketsizlik sonrası uyku modu** vardır ve 2026 ortasında gelen güvenlik kısıtlamaları nedeniyle harici ağ/SDK kullanımlarında ek doğrulamalar getirilmiştir (HF Community, Temmuz 2026).
- **Render ($0 Free vs. $7/ay Starter):** Render'ın ücretsiz web servisi 512 MB RAM ve aylık 750 saat sunar ancak **15 dakika trafik gelmezse kapanır (spin-down)** ve tekrar açılması **50–60 saniye** sürer (Render Docs; Temps Render Pricing, Ekim 2026). 7/24 uyanık kalması için Starter planı **$7/ay**'dır (Livemy, Ağustos 2026).
- **Fly.io (~$1.94 – $3.50/ay):** Artık yeni hesaplara kalıcı ücretsiz katman vermemekte, saniye bazlı faturalandırma uygulamaktadır. Ancak 7/24 açık kalan en küçük `shared-cpu-1x` (256 MB RAM) makine ayda yalnızca **~$1.94**, Streamlit/Panel için önerilen 512 MB RAM'li makine ise **~$3.19/ay** tutmaktadır (OfficeBooks Fly.io Review, Ağustos 2026; Kuberns, Nisan 2026).

- **Canlı Fiyat ve Mobil Kullanım Sorunu:** Streamlit'in çalışma modeli her veri güncellemesinde Python betiğini yukarıdan aşağıya yeniden çalıştırmaya (script rerun) veya `st.fragment(run_every="5s")` kullanmaya dayanır. Telefonda hücresel ağ (4G/5G) geçişlerinde Streamlit'in dahili WebSocket bağlantısı koptuğunda ekran grileşir ve tüm sayfa baştan yüklenir. Bu konuda **Plotly Dash** (`dcc.Interval` + istemci taraflı `clientside_callback`) ve **HoloViz Panel** (Tornado/Bokeh asenkron WebSocket) Streamlit'ten çok daha kararlıdır (Johal.in, Mayıs 2026).



Şekil 1: Sol panelde 2026 itibarıyla 7 farklı dağıtım mimarisinin aylık sabit bulut maliyeti (USD) ile tek kişilik bakım yükü (saat/ay) karşılaştırılmakta; sağ panelde ise ücretsiz katmanlardaki uyku modu (hibernation / spin-down) kaynaklı ilk açılış gecikmeleri gösterilmektedir (Temps, Ekim 2026; OfficeBooks, Ağustos 2026).

2.3. Seçenek (c): FastAPI + SQLite / PostgreSQL + Ön Yüz (Tam Yığın Mikro-Sunucu Mimarisi)

Tam yığın mimaride arka uçta asenkron Python **FastAPI** çalışır; veriler ilişkisel bir veritabanında (**SQLite WAL modu** veya yönetilen **PostgreSQL**) tutulur ve ön yüzde React/Svelte tabanlı bir arayüz REST/WebSocket üzerinden FastAPI ile haberleşir.

- **Veritabanı Seçimi (SQLite vs. Yönetilen Postgres):**
  - **Render Ücretsiz Postgres Tuzağı:** Render'ın ücretsiz PostgreSQL veritabanı **oluşturulduktan 30 gün sonra tamamen silinir (expires after 30 days)** (Livemy Render Pricing, Ağustos 2026); bu nedenle algoritmik işlem geçmişi için kesinlikle kullanılmamalıdır.
  - **Neon veya Supabase (Ücretsiz Sunucusuz Postgres — $0/ay):** Neon Free Plan proje başına **0.5 GB depolama** ve aylık **100 CU-saat (Compute Unit hours)** sunucusuz Postgres sağlar; sorgu olmadığında otomatik uyur ve ~1–1.5 saniyede uyanır (Neon Blog, Eylül 2025). Supabase ise 500 MB ücretsiz Postgres sunar ancak 7 gün hareketsizlikte projeyi duraklatır.
  - **Fly.io üzerinde FastAPI + SQLite (WAL) + Litestream (~$2.10/ay):** Tek kişilik bir sistem için harici bir Postgres sunucusu yönetmek yerine, FastAPI ile aynı `shared-cpu-1x` (256 MB RAM, $1.94/ay) sanal

makinedeki kalıcı diskte (1 GB Fly Volume = $0.15/ay) **SQLite (Write-Ahead Logging — WAL modu)** kullanmak çok daha hızlıdır (ağ gecikmesi 0 ms) ve Cloudflare R2'ye (10 GB ücretsiz) **Litestream** ile saniyelik yedeklenir.

- **Canlı Fiyat ve Kimlik Doğrulama Gücü:** FastAPI'nin `asyncio` döngüsü borsaların **Özel Kullanıcı Veri Akışlarına (Binance User Data Stream `listenKey`, VİOP FIX/WebSocket)** 7/24 bağlı kalabilir ve `HTTPOnly` güvenli çerezler + **OAuth2 / JWT (veya Passkeys/WebAuthn)** ile tam kurumsal koruma sağlar.
- **Neden Başlangıç İçin Erkendir? (Bakım Yükü):** Veritabanı şema göçleri (Alembic), Docker imajı güncellemeleri, arka plan WebSocket yeniden bağlanma (reconnection) mantığı ve hem arka uç hem ön yüz kodunu aynı anda sürdürmek tek bir kişi için ayda **3.5 – 4.5 saatlik sürekli DevOps yükü** yaratır. Şu an veriniz zaten GitHub Actions tarafından birkaç saatte bir üretildiği için, 7/24 çalışan bir FastAPI sunucusu kurmak başlangıç aşamasında **aşırı mühendisliktir (over-engineering)**.

2.4. Seçenek (d): PWA (Progressive Web App) Olarak Telefona Kurulum

PWA tek başına bir arka uç altyapısı değil, Seçenek (a), (b) veya (c)'deki web panosunun kök dizinine bir `manifest.json` ve bir `sw.js` (Service Worker) eklenerek telefonun ana ekranına **yerel uygulama (native app)** gibi kurulmasını sağlayan standarttır (CoderCops PWA 2026 Raporu, Mayıs 2026).

- **Maliyet ve Dağıtım Avantajı:** Apple App Store ($99/yıl geliştirici ücreti + katı kripto/finansal uygulama inceleme redleri) veya Google Play Store süreçlerine hiç girmeden **$0 maliyetle** dağıtılır.
- **2026 İtibarıyla iOS (Safari) ve Android Yetenekleri ve Kısıtları:**
  - **Android (Chrome):** Tam kurulum istemi (install prompt), arka plan senkronizasyonu (Background Sync) ve kesintisiz Web Push bildirimlerini destekler.
  - **iOS (Safari 16.4+ / iOS 18+):** Apple, iOS 16.4'ten itibaren **Ana Ekrana Eklenen (Add to Home Screen)** PWA'lar için **Web Push API** bildirimlerini desteklemektedir (Qvik, 2023; CoderCops, Mayıs 2026). Ancak iOS'ta kritik kısıtlar devam etmektedir: (1) Kullanıcı uygulamayı Safari'de açıp *Paylaş → Ana Ekrana Ekle* adımını manuel yapmadan push bildirimi alamaz, (2) Uygulama arka plana atıldığında iOS birkaç saniye içinde WebSocket bağlantısını dondurur, (3) `Cache-First` Service Worker yanlış kurgulanırsa yatırımcı ekranda saatler öncesinin bayat portföy verisini görebilir (Reddit PWA Analizi, Eylül 2025).
- **Algoritmik Trading İçin Doğru Service Worker Kuralı:** Statik UI dosyaları (JS/CSS/Font) için `Stale-While-Revalidate` kullanılırken, `portfolio.json` ve canlı fiyat API çağrıları için **kesinlikle `Network-Only` (veya çevrimdışı uyarılı `Network-First`)** stratejisi zorunlu tutulmalıdır; finansal panoda bayat önbellek (cached stale price) göstermek tehlikelidir.

2.5. Seçenek (e): Telegram Bot Arayüzü (Sinyal + "Uyguladım / Atladım" Butonu ve Mini App)

Tek kişilik algoritmik trading operasyonlarında **en yüksek fayda/maliyet oranına sahip operasyonel katman** Telegram Bot arayüzüdür (Telegram Bot API Docs). GitHub Actions iş akışınız zaten birkaç saatte bir çalışıp JSON ürettiği anda, aynı Python betiğinin sonunda `https://api.telegram.org/bot<TOKEN>/sendMessage` uç noktasına basit bir HTTP POST isteği atarak sinyali ve portföy özetini cebinize gönderebilirsiniz.

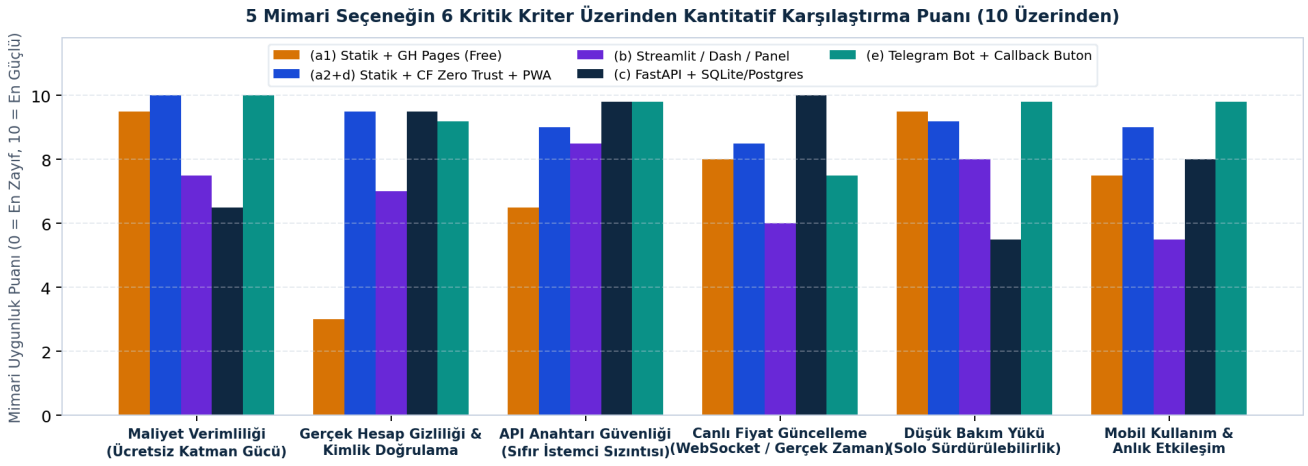
- **"Uyguladım / Atladım" Butonu Nasıl Çalışır? (Sunucusuz Webhook Mimarisi):**
  GitHub Actions sinyali gönderirken mesajın altına `reply_markup` içinde bir `InlineKeyboardMarkup` ekler: `[ ✅ Uyguladım (Filled)]` ve `[⏭️ Atladım (Skipped)]`. Ancak GitHub Actions işi bitip kapandığı için, siz 10 dakika sonra telefondaki butona bastığınızda Telegram'ın göndereceği `callback_query` paketini karşılayacak 7/24 uyanık bir uç noktaya ihtiyaç vardır. Bunun için 7/24 sunucu kiralamaya gerek yoktur: **Cloudflare Workers (Ücretsiz katman: günde 100,000 istek, $0/ay)** üzerinde çalışan 35 satırlık bir sunucusuz fonksiyon Telegram Webhook'u olarak kaydedilir (Serverless Telegram Webhook; Cloudflare Workers + D1).

  Siz telefondan "✅ **Uyguladım**" butonuna bastığınız anda:

  1. Telegram, Cloudflare Worker'a `X-Telegram-Bot-Api-Secret-Token` başlığıyla `callback_query` gönderir.

2. Worker, `from.id` değerinin sizin kişisel Telegram `chat_id`'niz olduğunu doğrular (başkası botu bulsa bile komut çalıştıramaz).
3. Worker, kararı ya **Cloudflare D1 (ücretsiz sunucusuz SQLite) / KV** tablosuna yazar ya da doğrudan GitHub REST API'sini (`POST /repos/{owner}/{repo}/dispatches`) tetikleyerek kararınızı deponuzdaki `decisions.json` dosyasına işler ve Telegram mesajını "✅ *26.10.2026 14:05'te Uygulandı olarak kaydedildi*" şeklinde günceller (`editMessageText`).

- **Telegram Mini Apps (TMA) Köprüsü:** Telegram Bot API'si ayrıca sohbet içindeki bir butona (`WebAppInfo`) basıldığında, Seçenek (a)'da hazırladığınız statik React/Svelte panosunu **doğrudan Telegram uygulamasının içinde tam ekran bir çekmece olarak (Telegram Mini App)** açmanıza ve kullanıcının kimliğini `window.Telegram.WebApp.initData` HMAC-SHA256 imzasıyla otomatik doğrulamanıza olanak tanır!



Şekil 2: Beş aday altyapı mimarisinin maliyet verimliliği, gerçek hesap gizliliği, API anahtarı güvenliği, canlı fiyat güncellemesi, düşük bakım yükü ve mobil kullanım kriterleri üzerinden kantitatif karşılaştırması. Statik Site + Cloudflare Zero Trust + PWA (a2+d) ile Telegram Bot (e) kombinasyonu tüm kriterlerde 8.5–10 bandında kalarak birbirini kusursuz tamamlamaktadır.

---

3. Altı Kritik Kriter Üzerinden Kapsamlı Karşılaştırma Matrisi (2026)

Aşağıdaki tablo, beş mimari seçeneği (ve GitHub Pages ile Cloudflare Pages arasındaki kritik güvenlik farkını) talep edilen altı değerlendirme boyutu üzerinden yan yana özetlemektedir:

Kriter	(a1) Statik Site + GitHub Pages	(a2 + d) Statik Site + CF Pages + PWA	(b) Streamlit / Dash / Panel	(c) FastAPI + SQLite/Postgres	(e) Telegram Bot + Callback Buton
1. Aylık Maliyet & Ücretsiz Sınırlar	**$0/ay** (Public repo) veya **$4/ay** (GH Pro). 1 GB depo, 100 GB bant genişliği (GH Docs).	**$0/ay**. Sınırsız bant genişliği, 500 build/ay, 50 kullanıcıya kadar ücretsiz Zero Trust (CF Access).	**$0/ay** (Streamlit 1 özel app / HF 16GB RAM ama uyku modlu); 7/24 uyanık: **$3.19/ay** (Fly) veya **$7/ay** (Render).	**$2.10/ay** (Fly.io 256MB + SQLite) veya **$7/ay** (Render + Neon 0.5GB Free Postgres).	**$0/ay**. Telegram API ücretsiz + Cloudflare Workers (100k istek/gün) + D1 (5 GB) ücretsiz.

Kriter	(a1) Statik Site + GitHub Pages	(a2 + d) Statik Site + CF Pages + PWA	(b) Streamlit / Dash / Panel	(c) FastAPI + SQLite/Postgres	(e) Telegram Bot + Callback Buton
2. Kimlik Doğrulama (Gerçek Hesap Verisi)	**Çok Zayıf:** Yerleşik giriş ekranı yoktur; URL'yi bilen herkes görür. İstemci taraflı AES-256 şifreleme şarttır.	**Kurumsal (10/10):** Cloudflare Access (Google/GitHub SSO veya E-posta OTP) dosyaları CDN katmanında korur.	**Orta–İyi:** Streamlit Cloud'da Google/GitHub izni; kendi sunucunuzda `streamlit-authenticator` veya Basic Auth.	**Kurumsal (10/10):** OAuth2 / JWT, `HTTPOnly` çerezler, Passkeys (WebAuthn) veya Cloudflare Tunnel.	**Çok Güçlü (9.5/10):** Telegram 2FA + Worker içinde `chat_id` beyaz listesi + `secret_token` doğrulaması.
3. Canlı Fiyat Güncellemesi	**İyi (İstemci WS):** Tarayıcıdan doğrudan Binance Public WebSocket'e bağlanır; VİOP/FX için REST polling.	**Çok İyi:** Tarayıcı Public WebSocket + Cloudflare Worker proxy üzerinden CORS takılmadan hızlı fiyat çekimi.	**Orta:** `st.fragment` ile 3–5 sn yenileme; ancak mobil ağ geçişlerinde WS kopması ve 25–60 sn cold-start sorunu.	**Mükemmel (10/10):** Sunucu 7/24 hem Public hem Private (bakiye/emir) WebSocket akışlarını dinler ve yayınlar.	**Olay Güdümlü:** Sinyal anında (0.2 sn) push bildirim; sohbet içinde `/durum` komutuyla anlık fiyat/PnL sorgusu.
4. Tek Kişilik Bakım Yükü	**~0.5 saat/ay** (Sıfır sunucu, sadece statik derleme).	**~0.8 saat/ay** (Sıfır sunucu, otomatik Git dağıtımı).	**~1.8 – 2.2 saat/ay** (Python paket/bağımlılık ve uyku modu yönetimi).	**~3.8 – 4.5 saat/ay** (Docker, DB göçleri, API + UI senkronizasyonu).	**~0.4 saat/ay** (Sıfır ön yüz, değişmeyen Bot API standardı).
5. API Anahtarlarının Güvenliği	**Yüksek:** Borsa anahtarları yalnızca GitHub Secrets'ta kalır; tarayıcıya asla gönderilmez.	**Çok Yüksek:** Borsa anahtarları GitHub Secrets ve CF Worker ortam değişkenlerinde (encrypted) tutulur.	**Yüksek:** Sunucu taraflı `st.secrets` veya ortam değişkenlerinde tutulur; istemciye sızmaz.	**En Yüksek:** Sunucu ortam değişkenlerinde tutulur; IP kısıtlamalı borsa API anahtarı (Static Egress IP) kullanılabilir.	**Çok Yüksek:** Borsa anahtarları GitHub Secrets'ta, Bot Token ise CF Worker Secret'ta izole tutulur.
6. Mobil Kullanım Deneyimi	**İyi:** Mobil tarayıcıda hızlı açılır ancak bildirim gönderemez.	**Çok İyi (9/10):** PWA ile ana ekranda tam ekran ikon, anlık açılış, iOS/Android Web Push desteği.	**Zayıf–Orta (5.5/10):** Çok sütunlu tablolar mobilde sıkışır; uykudan uyanma süresi mobilde kullanışsızdır.	**Çok İyi:** Özel tasarlanmış mobil ön yüz (PWA ile birleştirilebilir).	**Mükemmel (9.8/10):** Cebinizdeki en hızlı yerel uygulama; kilit ekranından tek tıkla "Uyguladım/Atladım" onayı.

4. Mimari Sentez: "Başlangıç İçin Öneri" ve "İleride Aşamalı Geçiş Yolu"

4.1. Başlangıç İçin Altın Standart Öneri ($0 / Ay, Sıfır Sunucu Bakımı): Hibrit "(e) Telegram Bot + (a2/d) Cloudflare Zero Trust Statik PWA" Mimarisi

Veriniz halihazırda GitHub Actions tarafından birkaç saatte bir JSON olarak üretildiği için, 2026 itibarıyla tek bir kişinin bakımını yapacağı en rasyonel, sıfır maliyetli (**$0/ay**) ve en güvenli başlangıç mimarisi **tek bir seçeneğe sıkışmak değil, (e) ve (a+d) seçeneklerini iki katmanlı bir "Komuta ve Görselleştirme" yapısında birleştirmektir**:

1. **Katman 1 — Bildirim ve Eylem Katmanı (Seçenek e: Telegram Bot + Cloudflare Worker Webhook):**
   - GitHub Actions iş akışınız birkaç saatte bir çalışıp JSON dosyasını ürettiğinde, Python betiğinizin sonuna eklenecek 25 satırlık bir fonksiyonla yeni sinyalleri ve portföy özetini doğrudan kişisel Telegram hesabınıza gönderir.

- Mesajın altında `[✅ Uyguladım]`, `[⏭️ Atladım]` ve `[📊 Panoyu Aç (Mini App)]` butonları yer alır.
- Ücretsiz bir **Cloudflare Worker** (veya **Cloudflare D1** sunucusuz SQLite tablosu) buton tıklamalarını (`callback_query`) saniyesinde yakalar ve hangi sinyali uyguladığınızı/atladığınızı kaydeder. Bir sonraki GitHub Actions çalışması bu kararları okuyarak gerçek pozisyonlarınızı günceller.

2. **Katman 2 — Görsel Analiz ve Canlı Fiyat Katmanı (Seçenek a + d: Svelte/React Statik PWA + Cloudflare Pages & Zero Trust Access):**
   - Karmaşık grafikleri (özsermaye eğrisi, düşüş/drawdown grafiği, strateji korelasyon matrisi) incelemek istediğinizde, GitHub Actions'ın ürettiği `portfolio.json` dosyasını okuyan hafif bir **React/Vite veya Svelte** statik sitesini **Cloudflare Pages** üzerine dağıtırsınız.
   - Önüne **Cloudflare Zero Trust Access** (50 kullanıcıya kadar ücretsiz) koyarak sadece kendi e-posta adresinize izin verirsiniz; böylece gerçek hesap bakiyeniz %100 korunur.
   - Siteye `manifest.json` ekleyerek telefonunuza **PWA** olarak (veya Telegram içinden **Telegram Mini App** olarak) kurarsınız. Sayfa açıldığında pozisyon büyüklüklerini JSON'dan alırken, tarayıcıdan doğrudan **Binance Public WebSocket**'ine bağlanarak anlık kâr/zararı saniyelik olarak canlı günceller.

```
# Başlangıç Mimarisi İçin Örnek Akış: GitHub Actions (Python) -> Telegram Inline Buton + Cloudflare Worker
import os, json, requests

BOT_TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]   # GitHub Actions Secret'ta güvende
CHAT_ID   = os.environ["TELEGRAM_CHAT_ID"]     # Sadece sizin kişisel chat_id'niz
PWA_URL   = "https://quant-pano.pages.dev"     # Cloudflare Zero Trust korumalı statik PWA

def send_signal_with_action_buttons(signal_id: str, symbol: str, side: str, price: float, weight_pct: float):
    text = (
        f"🔔 *YENİ ALGORİTMİK SİNYAL*
"
        f"• *Sembol:* `{symbol}` | *Yön:* *{side}*
"
        f"• *Referans Fiyat:* `{price}` | *Hedef Ağırlık:* `%{weight_pct:.1f}`
"
        f"• *Sinyal ID:* `{signal_id}`"
    )
    payload = {
        "chat_id": CHAT_ID,
        "text": text,
        "parse_mode": "Markdown",
        "reply_markup": {
            "inline_keyboard": [
                [
                    {"text": "✅ Uyguladım (Filled)", "callback_data": f"EXEC:{signal_id}:{price}"},
                    {"text": "⏭️ Atladım (Skipped)",  "callback_data": f"SKIP:{signal_id}:0"}
                ],
                [
                    {"text": "📊 Canlı Portföy Panosunu Aç (PWA)", "web_app": {"url": PWA_URL}}
                ]
            ]
        }
    }
    resp = requests.post(f"https://api.telegram.org/bot{BOT_TOKEN}/sendMessage", json=payload, timeout=5)
    return resp.json()
```

4.2. İleride Aşamalı Geçiş Yolu (Evrimsel Yol Haritası: Faz 1 → Faz 2 → Faz 3)

Sisteminiz büyüdükçe, strateji frekansı saatlikten dakikalık/saniyelik düzeye indikçe veya BIST/VİOP ile Binance arasında gerçek zamanlı otomatik emir iletimine (auto-execution) geçtiğinizde mevcut kodunuzu çöpe atmadan izleyeceğiniz **3 Fazlı Geçiş Yolu** aşağıda tanımlanmıştır:

Evrim Fazı	Ne Zaman Geçilmeli? (Tetikleyici İhtiyaç)	Altyapı Mimarisi	Maliyet ve Geçiş Eforu
Faz 1: Başlangıç (Bugün — Sıfır Sunucu)	Veri birkaç saatte bir GitHub Actions'ta JSON olarak üretiliyor; sinyaller yarı-otomatik onaylanıyor.	**GitHub Actions (JSON) + Cloudflare Pages (React/Svelte PWA + Zero Trust Auth + Tarayıcı Public WS) + Telegram Bot (CF Worker + D1 Webhook)**	**$0 / ay** Kurulum: 1–2 hafta sonu Bakım: ~0.8 saat/ay
Faz 2: Orta Evre (İlişkisel Tarihçe & Analitik)	JSON dosyası 5–10 MB'ı aştığında, binlerce geçmiş işlem üzerinde SQL sorgusu ve kayma (slippage) analizi gerektiğinde.	Faz 1'deki ön yüz (PWA) ve Telegram Bot **aynen korunur**; yalnızca JSON yerine **Cloudflare D1 (SQLite)** veya **Neon Serverless Postgres (Free 0.5 GB)** bağlanır. İsteğe bağlı olarak derin analitik için **HF Spaces / Fly.io üzerinde özel bir HoloViz Panel / Streamlit** sayfası eklenir.	**$0 – $3.19 / ay** Ön yüz değişmez; sadece veri kaynağı JSON'dan SQL API'ye yönlendirilir.
Faz 3: İleri Evre (7/24 Otomatik İcra & Özel WS)	Saatlik GitHub Actions yetmediğinde; 7/24 saniyelik sinyal üretimi, özel bakiye WebSocket'i ve Kill Switch gerektiğinde.	**Fly.io (`shared-cpu-1x`, 512MB RAM)** veya **Hetzner Cloud VPS** üzerinde 7/24 çalışan **FastAPI + SQLite (WAL + Litestream R2 yedekleme)**. Faz 1'deki React/Svelte PWA ve Telegram Bot doğrudan FastAPI uç noktalarına bağlanır.	**~$3.50 – $5.00 / ay** Faz 1'deki PWA ve Telegram arayüzü %100 korunarak arka uç GitHub Actions'tan FastAPI'ye taşınır.

4.3. Kaynakça ve Doğrulanmış Teknik Referanslar (2025–2026)

- **CoderCops (22 Mayıs 2026).** *Progressive Web Apps in 2026: What Works on iOS, Android*. iOS Safari Home Screen Web Push ve arka plan kısıtları analizi. Erişim: https://blog.codercops.com/blog/progressive-web-apps-2026
- **Display.dev & OOO.run (Haziran – Ekim 2026).** *Cloudflare Free Tier & Cloudflare Access Static Site Authentication (50 Users Free Zero Trust)*. Erişim: https://display.dev/blog/cloudflare-access-alternative-static-sharing ve https://ooo.run/en/post/cloudflare-free-tier.html
- **Fly.io & OfficeBooks (Ağustos 2026).** *Fly.io Resource Pricing & 2026 Review (shared-cpu-1x $1.94/month per-second billing)*. Erişim: https://fly.io/pricing/ ve https://officebook.dev/tools/fly-io/
- **GitHub Documentation (2026).** *GitHub Pages Limits, Plans, and Access Control for Private Sites*. Erişim: https://docs.github.com/en/pages/getting-started-with-github-pages/github-pages-limits ve https://docs.github.com/get-started/learning-about-github/githubs-products
- **Hugging Face Documentation & Codex (Mayıs – Temmuz 2026).** *Spaces Overview & Free CPU Basic (2 vCPU, 16 GB RAM) Limits*. Erişim: https://huggingface.co/docs/hub/en/spaces-overview ve https://medium.com/codex/how-hugging-face-spaces-is-a-solid-repository-for-data-analysis-2cbad68fbf67
- **Johal.in Engineering Survey (9 Mayıs 2026).** *The Ultimate Dashboard Showdown: Streamlit vs. Plotly Dash vs. HoloViz Panel Latency & Memory Benchmark*. Erişim: https://www.johal.in/ultimate-dashboard-showdown-pandas-review
- **Neon Serverless Postgres Documentation (Eylül 2025 – 2026).** *Neon Plans, Compute Unit Hours (100 CU-hours Free), and Auto-Suspend Architecture*. Erişim: https://neon.com/docs/introduction/plans ve https://neon.com/blog/how-to-make-the-most-of-neons-free-plan
- **Render Documentation & Temps / Livemy (Ağustos – Ekim 2026).** *Deploy for Free on Render: 15-Minute Idle Spin-Down and 30-Day Free Postgres Expiration*. Erişim: https://render.com/docs/free ve https://temps.sh/blog/render-pricing-2026
- **Streamlit Documentation & Community Cloud (2025 – Ekim 2026).** *Manage Your App, Private App Quota, and App Hibernation (Sleep Mode) Rules*. Erişim: https://docs.streamlit.io/deploy/streamlit-community-

cloud/manage-your-app ve https://studentandstartupdeals.danubelabs.net/providers/streamlit-community.html

- **Telegram Bot API & Serverless Webhook Architecture (2025 – 2026).** *InlineKeyboardMarkup, CallbackQuery, and Cloudflare Workers Webhook Deployment*. Erişim: https://classic.yarnpkg.com/en/package/node-telegram-bot-api ve https://github.com/furic/richfolio

4.4. Cloudflare Worker Webhook ve D1 Karar Kayıt Kodu (Uyguladım / Atladım Buton İşleyicisi)

Telegram üzerindeki `[✅ Uyguladım]` ve `[⏭️ Atladım]` butonlarına basıldığında sunucusuz (serverless) olarak çalışan ve kararı hem **Cloudflare D1 (SQLite)** veritabanına kaydeden hem de Telegram mesajını anında güncelleyen örnek **Cloudflare Worker** kodu aşağıda verilmiştir (Serverless Telegram Webhook; Cloudflare Workers + D1):

```
// Cloudflare Worker (worker.js) - $0/ay Ücretsiz Katman (Günde 100.000 İstek)
export default {
  async fetch(request, env) {
    // 1. Güvenlik: Telegram Secret Token başlığını doğrula
    const secret = request.headers.get("X-Telegram-Bot-Api-Secret-Token");
    if (secret !== env.TELEGRAM_WEBHOOK_SECRET) {
      return new Response("Unauthorized", { status: 403 });
    }
    const update = await request.json();
    if (!update.callback_query) return new Response("OK", { status: 200 });

    const cb = update.callback_query;
    // 2. Kimlik Doğrulama: Sadece tek kişilik sahip (owner chat_id) buton kullanabilir
    if (String(cb.from.id) !== String(env.ALLOWED_CHAT_ID)) {
      return new Response("Forbidden User", { status: 403 });
    }

    // callback_data formatı: "EXEC:SIG_20261026_BTC:88450" veya "SKIP:SIG_20261026_BTC:0"
    const [action, signalId, refPrice] = cb.data.split(":");
    const timestamp = new Date().toISOString();

    // 3. Kararı Cloudflare D1 (SQLite) tablosuna kaydet
    await env.DB.prepare(
      "INSERT INTO trade_decisions (signal_id, action, ref_price, decided_at) VALUES (?, ?, ?, ?)"
    ).bind(signalId, action, parseFloat(refPrice), timestamp).run();

    // 4. Telegram mesajındaki butonları kaldır ve durumu güncelle
    const statusBadge = action === "EXEC" ? "✅ UYGULANDI (FILLED)" : "⏭️ ATLANDI (SKIPPED)";
    await fetch(`https://api.telegram.org/bot${env.TELEGRAM_BOT_TOKEN}/editMessageText`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        chat_id: cb.message.chat.id,
        message_id: cb.message.message_id,
        text: `${cb.message.text} | Karar: ${statusBadge} (${timestamp})`,
        parse_mode: "Markdown"
      })
    });
    return new Response("OK", { status: 200 });
  }
};
```

Mimari Sonuç Özeti: Tek kişilik algoritmik yatırım sistemlerinde en büyük hata, daha ilk günden karmaşık bir Kubernetes/Docker veya çok katmanlı FastAPI + PostgreSQL + React yığını kurarak strateji araştırmasına ayrılacak zamanı DevOps bakımına harcamaktır. **GitHub Actions (JSON) + Cloudflare Pages (Zero Trust korumalı Svelte/React PWA + Tarayıcı Public WebSocket) + Telegram Bot (Cloudflare Worker + D1 "Uyguladım/Atladım" Webhook)** üçlüsü, 2026 itibarıyla **$0/ay maliyet**, **kurumsal düzeyde sıfır-güven kimlik doğrulaması**, **sıfır API anahtarı sızıntısı** ve **ayda 1 saatin altında bakım yükü** ile en rasyonel başlangıç mimarisidir.